
Self-Harnessing Recursive Language Models

Abstract

Recursive language models (RLMs) let a bounded-context model operate over inputs larger than its context window by offloading the input, decomposing it, invoking recursive model calls, and aggregating their outputs [1]. Vanilla RLMs suffer from many failure cases, necessitating the fine-tuning LLMs and/or engineering harnesses by hand to overcome their limitations. We propose adapting the SELF-HARNESS framework of Zhang et al. [2] to RLMs, so that a fixed model improves its own recursive harness from evidence produced while running: it mines recurring failure mechanisms from verifier-grounded execution traces, proposes minimal edits to declared harness surfaces, and promotes only edits that survive non-regressive validation. The optimized harness is frozen and evaluated on instances $8\text{--}32\times$ longer than any seen during optimization and in new target environments. Results forthcoming.

1 Introduction

Recursive language models (RLMs) reframe inference as a program in which a language model inspects its own context, decomposes it, and invokes fresh copies of itself on the pieces [1]. Every call stays inside the native context window, letting a bounded-context model reason over prompts one to two orders of magnitude larger than that window. The mechanism behind these gains is distributional rather than architectural: the harness shapes each call so that a task which is out-of-distribution as a whole is handled through calls that are individually in-distribution [3].

Performance therefore depends not only on the model but on the harness that determines when and how recursion occurs. However, the RLM harness grants the capacity to recurse without guaranteeing good recursion. Documented failures are recurring and model-specific: policies collapse the entire problem into a single sub-call [3], deeper recursion degrades accuracy while inflating cost [4], and recursion halts by heuristic rather than by evidence that further work would not help [5]. Yet today, models are fine-tuned to function within the RLM harness more effectively [3, 6], or the RLM harness is engineered by hand [7–9]. An alternative is to let the harness optimize itself: the model inspects its own execution trajectories and proposes edits to the scaffold that produced them [2, 10, 11]. Because the evidence driving each edit comes from the deployed model on the deployed task distribution, the resulting harness can adapt both to the structure of the task and to model-specific capabilities, a fit that no single hand-designed harness appears to provide universally [2, 11, 12].

We ask whether a fixed language model can instead improve its own RLM harness from evidence produced during execution:

Can a language model identify recurring failures in its own recursive behavior, convert them into targeted harness edits, and discover an orchestration harness that transfers across input lengths and task environments?

2 Contributions

Our contributions are:

1. **SELF-HARNESS for recursive language models.** We adapt SELF-HARNESS to RLMs by declaring nine editable harness surfaces keyed to phases of the RLM turn loop: the REPL contract, the decomposition, execution, verification and recovery instructions, the runtime policy, the metadata function, the REPL helpers, and the answer middleware (fig. 1). A fixed model mines recurring failure mechanisms from structured summaries of its recursive traces, proposes targeted edits, and validates them without weight updates or a stronger external model. Optimization starts from a deliberately sparse initial harness in which seven of the nine surfaces are empty, disabled, or a single generic line.
2. **Evidence of length generalization.** We optimize only on short source instances, freeze the harness, and evaluate it on source instances $8\text{--}32\times$ longer, testing whether self-discovered harness improvements transfer across input length.
3. **Evidence of cross-environment transfer.** We apply the same frozen harness to the short and long splits of an unseen target environment. This tests whether the recovered strategy transfers beyond the environment that produced it and whether it survives simultaneous changes in task environment and input length.

3 Methods

3.1 Model and Harness

We use a single frozen language model, with the same model and decoding configuration serving the root and all recursive sub-calls. Three invariants of the initial reference RLM implementation of Zhang et al. [1], remain fixed.

1. **Prompt-as-variable.** The prompt lives in the environment as a REPL variable and is never copied into the root context.
2. **Programmatic sub-calls.** Sub-calls are issued by code, in loops, over slices of the prompt.
3. **Outputs-in-variables.** The final answer is accumulated in REPL variable and returned from it.

Their runtime is extended with four injection points: an injected metadata function, answer-protocol middleware, sub-call retry and validation, and enforceable batch caps. Each defaults to the shipped behavior, so an unconfigured harness is byte-identical to the reference implementation. These are the seams through which the declared surfaces act; model weights, external tools, and the evaluator remain fixed.

Following Zhang et al. [2], whose initial harness declares nine builder functions as configuration points of the agent loop, we declare **nine editable surfaces keyed to phases of the RLM turn loop**, each a builder function in a single harness-definition module. Figure 1 shows the initial harness H_0 in full: each surface with the value it returns at the starting point of optimization (the condition reported as B1 in section 3.4).

Declaring the surfaces from the loop’s phase structure, rather than selecting them from failures already documented for RLMs, is deliberate. A surface set chosen by reading a published catalogue of failures would make it impossible at analysis time to separate what the optimization loop discovered from what its designer already knew. Surface *selection* is part of the fixed scaffold shared by every condition, exactly as it is in Zhang et al. [2]: the model proposes bounded edits to declared surfaces and does not choose which surfaces exist.

H_0 is deliberately sparse. Seven of its nine surfaces return nothing, a disabled policy, or a single generic line, so the orchestration guidance a capable harness eventually carries is guidance the loop has to recover from its own traces rather than reread from its own system prompt.

3.2 Environments and splits.

We evaluate on two environments from the length-generalization suite of Zhang and Khattab [3].

- **GraphWalks** [13], multi-hop reachability and traversal queries over a graph presented in context. A sub-call asking for reachability within an induced subgraph has a checkable answer, so a child can be verified directly.

```

1  # shrlm/rlm_harness.py: the nine surfaces Self-Harness may edit (S1 to S9).
2
3  MINIMAL_REPL_CONTRACT = """
4  You are a Recursive Language Model (RLM): a language model whose prompt-related
5  information lives in a Python REPL that you drive turn by turn.
6  Available in the REPL:
7  - 'context': the information related to the prompt ('str' or 'list[str]').
8  - 'llm_query(prompt, model=None) -> str'; 'llm_query_batched(prompts, ...)'
9  - 'rlm_query(...)' / 'rlm_query_batched(...)': recursive RLM sub-calls.
10 - 'SHOW_VARS()' -> str': list every variable currently in the REPL.
11 - 'answer': dict {"content": "", "ready": False}; set both to submit.
12 {custom_tools_section}
13 Write code in ''repl'' blocks, one per turn; the REPL persists. Only
14 'print(...)' output is shown back. Outputs over ~20K characters are truncated.
15 """
16
17 def build_repl_contract() -> str:                                # S1
18     return MINIMAL_REPL_CONTRACT
19
20 def build_decomposition_instruction() -> str:                    # S2
21     return ("Start by probing 'context' to understand what you have, "
22            "then decide how the task breaks into steps.")
23
24 def build_execution_instruction() -> str:                        # S3
25     return ("On each turn, run one block of code, print what you need "
26            "to see, and build the result up in REPL variables.")
27
28 def build_verification_instruction() -> str:                     # S4
29     return 'Check your answer before setting 'answer["ready"] = True'.'
30
31 def build_recovery_instruction() -> str:                         # S5
32     return ("If a sub-call errors or returns something you cannot use, "
33            "adjust the approach and try again.")
34
35 def build_runtime_policy() -> dict[str, Any]:                  # S6
36     return {
37         "enabled": False,
38         "max_prompt_chars": None,
39         "max_batch_width": None,
40         "max_depth": None,
41         "retry_on_syntax_error": None,
42         "max_retries": None,
43         "validate_sub_output": None,
44     }
45
46 def build_metadata(                                             # S7
47     stdout: str,
48     repl_inventory: dict[str, tuple[str, int]],
49     max_character_length: int = DEFAULT_MAX_CHARACTER_LENGTH,
50 ) -> str:
51     return default_metadata_builder(stdout, repl_inventory,
52                                     max_character_length)
53
54 def build_repl_helpers() -> dict[str, Any]:                    # S8
55     return {}
56
57 def build_sub_repl_helpers() -> dict[str, Any]:               # S8
58     return {}
59
60 def build_answer_middleware() -> AnswerMiddlewareFn:          # S9
61     return accept_answer

```

Figure 1: Initial harness H_0 and the editable interface used as the starting point for SELF-HARNESS: nine declared surfaces, one builder function each, keyed to phases of the RLM turn loop. The harness is intentionally kept sparse, with seven of the nine returning nothing, a disabled policy, or a single generic line, so that the orchestration strategy a tuned harness carries is something the optimization loop must recover from its own execution traces. S6, S7 and S9 required a runtime injection point in the reference implementation of Zhang et al. [1]; the rest are reached through its existing prompt and tool configuration. A proposal modifies exactly one surface.

- **OOLONG-Pairs**, a modified variant of OOLONG [14] that asks for pairs of elements satisfying a given constraint. A candidate pair either satisfies the constraint or does not, which is checkable in isolation.

Both require decomposing an input into independently solvable units and aggregating their outputs, but they differ in surface task and data distribution, which is the preregistered structural criterion by which they were chosen. Each provides matched short and long instances, with long inputs approximately 8–32× larger, and a deterministic verifier that we use unmodified. One is preregistered as the source environment and the other serves as the target.

Both environments were selected because they additionally admit *synthesized sub-verifiers*: the sub-problems a decomposition produces are themselves checkable without reference to the root answer. This is what makes the failure attribution of section 3.3 more than a model judgment. When the root answer is wrong, running the sub-verifier on each child separates a root failure, in which correct children were aggregated into a wrong answer, from a child failure, in which a sub-call returned a wrong local result that the root faithfully combined. The two implicate different editable surfaces, and without sub-verification the distinction rests on the proposer’s reading of a trace rather than on a checkable outcome.

Only the source environment is used for optimization. Its short split is partitioned into held-in, held-out, and test sets. Held-in traces support weakness mining, the held-out split serves only as the promotion gate, and the source-short test and source-long test remain inaccessible until the harness is frozen. No target-environment instances, traces, verifier outcomes, or aggregate results are exposed during optimization. The frozen harness is subsequently evaluated on both the short and long target splits.

3.3 Self-Harness Optimization

We adapt the SELF-HARNESS framework of Zhang et al. [2] to RLMs. The model weights remain fixed throughout, and the same model is used both to execute tasks and to propose harness changes. The editable surfaces are the nine declared in fig. 1, and a proposal modifies exactly one of them. The model may not modify the evaluator, external tools, or any of the three invariants of section 3.1. One further surface is permanently excluded: allowing the proposer to truncate or pre-summarize the prompt variable into root context. It would likely show held-in gains, and it violates the first invariant by turning the RLM back into a compaction agent. Each optimization round has three stages.

First, **Weakness Mining** runs the current harness on short held-in instances from a source environment and records verifier outcomes and recursive execution traces. Each sub-call is additionally scored by the environment’s synthesized sub-verifier, so a failed run carries a per-child correctness label alongside the root outcome. Failures are converted into a structured record containing the verifier-level failure, the level at which the error signal first appears, the agent behavior causally associated with it, the implicated harness mechanism, and supporting trace excerpts. Failures are then clustered by the verifier-grounded signature producing an evidence bundle of recurring, actionable failure patterns.

$$\phi(r_i) = (\text{verifier cause, failing level, causal status, agent mechanism}),$$

Second, **Harness Proposal** provides the same fixed model with the mined patterns, passing behaviors to preserve, and prior edit history, and asks it to generate several distinct, minimal candidate edits, excluding patterns not plausibly addressable by an editable surface. Each proposal must target one mined failure pattern, modify one declared harness surface, state its predicted behavioral effect, and identify possible regressions.

Third, **Proposal Validation** evaluates each candidate on both the source held-in split and a disjoint short held-out split that is never shown to the proposer. An edit is promoted only if it produces no meaningful accuracy regression and its sub-call/cost fall within a preregistered band. Unlike Zhang et al. [2], when multiple compatible edits pass, we re-evaluate the merged harness before promotion. Optimization stops after a fixed number of rounds or several consecutive rounds without a promotion.

3.4 Baselines and Evaluation

We evaluate the self-harnessed RLM against two baselines.

- **SH-RLM: Self-Harnessed RLM (ours)**. The frozen harness produced by section 3.3.
- **B1: initial RLM (H_0)**. The unmodified reference harness with every surface of fig. 1 at its H_0 default, and the starting point of optimization.
- **H1: λ -RLM [8]**. A hand-designed extension, carried over unmodified, that replaces free-form recursive code generation with a typed functional runtime and invokes the model

only on bounded leaf sub-problems. It measures SELF-HARNESS against human harness engineering rather than against an unmodified starting point.

One further condition relaxes the fixed-weight constraint and is reported separately.

- **F1: fine-tuned RLM.** The same backbone RL-trained inside the initial harness on the source short split, following Zhang and Khattab [3] (appendix A). It bounds how much of the residual gap after harness optimization is attributable to model capability rather than orchestration. Because weight training and harness optimization incur non-comparable costs, F1 is a reference point rather than a matched comparison, and is excluded from the primary comparisons.

Each condition is evaluated on four untouched categories of test set, with the two target categories reported separately for each target environment:

1. **source-short**, measuring improvement at the optimization length;
2. **source-long**, measuring length generalization;
3. **target-short**, measuring cross-environment transfer without a length shift; and
4. **target-long**, measuring cross-environment transfer under an additional length shift.

The primary comparisons are SH-RLM versus B1 on source-long, target-short, and target-long, with SH-RLM versus H1 on the same three sets indicating whether a self-discovered harness is competitive with a hand-designed one. Improvement on source-long indicates that the learned harness changes survive a length shift within the optimization environment. Improvement on target-short provides the cleanest evidence that the edits capture a transferable compositional strategy rather than a source-specific rule. Improvement on target-long tests whether that transfer remains effective when environment and input length change simultaneously.

3.5 Analysis

The primary metric is verifier accuracy, reported separately for all four test sets. Results are averaged over repeated seeded runs and accompanied by bootstrap confidence intervals over task instances. B1 and SH-RLM are compared using a paired per-instance test; the exact test and repetition-to-instance aggregation rule will be preregistered.

Secondary efficiency metrics are total input and output tokens, recursive-call count, maximum recursion depth, and accuracy per million tokens. Trace analysis tests whether optimization changes the mechanisms it was intended to repair. In particular, we report the frequency of each mined failure pattern before and after optimization and measure whole-input sub-call collapse, defined as a run that delegates most of the input to one child or performs no meaningful decomposition. Using the sub-verifiers, we also report the share of failures attributable to the root and to the children in each condition and split, which shows whether optimization moved errors down the call tree, repaired them, or merely relocated them.

Two ablations, one withholding the sub-verifier signal from the optimization loop and one removing each promoted edit from the final harness, are specified in appendix B. Additionally, we include an optional analysis on alignment drift in appendix C.

4 Feasibility and Cost

The SELF-HARNESS optimization loop runs only on the source environment. Let n_{in} and n_{ho} denote the source held-in and held-out sizes, m the number of mining repetitions, v the validation repetitions, and K the proposal width. Excluding occasional merged-candidate checks, each optimization round requires

$$R_{\text{round}} = m n_{\text{in}} + v(K + 1)(n_{\text{in}} + n_{\text{ho}})$$

runs. The first term collects source held-in traces for weakness mining; the second evaluates the current harness and K candidate harnesses on the two source optimization splits. Optimization is capped at T rounds and stops early after a preregistered number of rounds without promotion.

At $n_{in} = 24$, $n_{ho} = 40$, $m = 2$, $v = 4$, and $K = 4$, this is 1,328 runs per round and 19,920 short runs over $T = 15$ rounds (SELF-HARNESS used 15, 18, and 21, rounds for the three model families tested). Taking three passes over the stored context plus root overhead gives roughly 1.2×10^5 tokens for a typical short run, so optimization costs approximately 2.4×10^9 tokens.

After optimization, the three fixed-weight conditions B1, H1, and SH-RLM are evaluated on the source-short, source-long, target-short, and target-long test sets; F1 is budgeted separately (appendix A). The target environment therefore adds final-evaluation cost but no mining, proposal, or candidate-validation cost. Total cost can be expressed as

$$C_{\text{total}} = C_{\text{source optimization}} + C_{\text{source evaluation}} + C_{\text{target evaluation}}.$$

Evaluating three conditions on two environments at 40 short-test and 150 long-test instances with 3 repetitions is 720 short and 2,700 long runs. The long runs dominate: at inputs 8–32 $\times$ larger, they average roughly 1.2×10^6 tokens each, giving approximately 3.2×10^9 tokens for final evaluation against 9×10^7 for the short tests. The project total is therefore approximately $5\text{--}6 \times 10^9$ tokens. Served locally that is roughly 650 H100-hours; purchased as hosted inference at current open-weights rates it is approximately \$1,200–\$2,000. The sub-verification ablation of appendix B adds one further optimization run, or approximately 2.4×10^9 tokens, and is budgeted as contingent.

Cost is dominated by long-instance evaluation and by repeated reads of the offloaded context during recursion, and the passes-over-context multiplier is the dominant uncertainty: a factor-of-two error moves the total to between 3×10^9 and 1.1×10^{10} tokens. We will run a small pilot in both environments before optimization to measure tokens per run, recursive calls, and effective passes over the stored context, and these measurements will fix the final test sizes and token budget before the first harness edit is proposed. If needed, we can reduce long-test sample size in all four evaluation conditions to reduce cost.

5 Timeline

Completed work. Three components of the system are already built and working. Both evaluation environments are implemented with their deterministic verifiers and synthesized sub-verifiers integrated. The initial RLM is implemented with the nine editable harness surfaces of fig. 1 declared and their runtime injection points in place. The first stage of the SELF-HARNESS loop is operational: recursive execution traces are collected, converted into structured failure records, and clustered by verifier-grounded signature.

The remaining work is designed for completion over six weeks. Each week is split into three chunks that are independent of one another and can proceed in parallel; dependencies run only between weeks.

Week 1: Preregistration and baselines; begin the SELF-HARNESS loop; write introduction.

(i) Preregister the source and target environment assignment, task-structure criterion, model checkpoint, split sizes, evaluation repetitions, promotion rule, and inference budgets, and reproduce baseline behavior on a small sample. (ii) Begin the remaining loop stages on top of the working trace clustering: evidence-bundle construction and parallel proposal generation. (iii) Write the introduction.

Week 2: Pilot and calibration; continue the SELF-HARNESS loop; write related work. (i) Run pilot instances from both environments to estimate token cost, latency, context rereading, and baseline variance; define the promotion gate’s empirical noise margin from repeated source-short evaluations; finalize test sizes and apply any preregistered cost de-scoping. (ii) Implement candidate validation, merged-edit re-evaluation, early stopping, and harness versioning. (iii) Write the related work.

Week 3: Validate the loop; build evaluation tooling; write methods. (i) Test the complete loop on a small development subset excluded from the final experiment, and verify that runs are reproducible from saved configurations. (ii) Integrate the hand-designed λ -RLM baseline (H1) and build the evaluation and analysis tooling: matched-budget configurations, verifier-accuracy aggregation, confidence intervals, paired tests, and efficiency metrics. (iii) Write the methods section.

240 **Week 4: Source-environment optimization and freeze; write ablations and discussion.** (i) Run
 241 SELF-HARNESS on the source held-in and held-out splits, inspecting logs for infrastructure
 242 failures without manually changing proposed edits or promotion decisions, and freeze the
 243 final harness once the maximum-round or patience criterion is reached; no test split is
 244 opened before the freeze. (ii) Dry-run the four-way evaluation pipeline on the optimization
 245 splits so evaluation starts immediately at freeze, and run the optional alignment-drift probe
 246 across the accumulating harness lineage. (iii) Draft the ablations appendix and begin the
 247 discussion from optimization-split trace case studies.

248 **Week 5: Fixed-weight evaluation; analysis; write results.** (i) Run the four-way evaluation of B1,
 249 H1, and SH-RLM on the untouched test sets under matched inference budgets, with leave-
 250 one-edit-out ablations where budget permits and a published fine-tuned checkpoint if one is
 251 available. (ii) As results arrive, compute verifier accuracy, confidence intervals, paired com-
 252 parisons, token efficiency, recursion depth, call counts, and whole-input sub-call collapse.
 253 (iii) Write the results section.

254 **Week 6: Consolidation and release; write limitations and conclusion.** (i) Complete any remain-
 255 ing long-split evaluation runs and ablations. (ii) Compare the frequency of mined failure
 256 mechanisms before and after optimization and finish the trace case studies. (iii) Assemble
 257 the manuscript, write limitations and conclusion, and release the split identifiers, configu-
 258 rations, harness lineage, proposal and validation logs, and evaluation scripts.

259 The critical path runs through chunk (i) of each week: preregistration, pilot, loop validation, harness
 260 freeze, then evaluation on data that remains untouched during optimization. The remaining chunks
 261 are parallel engineering and writing tracks that keep the freeze date off the writing’s critical path.

References

- [1] Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models. *arXiv preprint arXiv:2512.24601*, 2025. URL <https://arxiv.org/abs/2512.24601>.
- [2] Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026. URL <https://arxiv.org/abs/2606.09498>.
- [3] Alex L. Zhang and Omar Khattab. Language model harnesses are compositional generalizers. Blog post, July 2026. URL <https://alexzhang13.github.io/blog/2026/harness/>. Accessed July 2026.
- [4] Daren Wang. Think, but don’t overthink: Reproducing recursive language models. *arXiv preprint arXiv:2603.02615*, 2026. URL <https://arxiv.org/abs/2603.02615>.
- [5] Debashis Guha, Amritendu Mukherjee, Sanjay Kukreja, and Tarun Kumar. State representation and termination for recursive reasoning systems. *arXiv preprint arXiv:2605.06690*, 2026. URL <https://arxiv.org/abs/2605.06690>.
- [6] Chenxiao Yang, Nathan Srebro, and Zhiyuan Li. Recursive models for long-horizon reasoning. *arXiv preprint arXiv:2603.02112*, 2026. URL <https://arxiv.org/abs/2603.02112>. ICML 2026.
- [7] Keivan Alizadeh, Parshin Shojaee, Minsik Cho, and Mehrdad Farajtabar. Recursive language models meet uncertainty: The surprising effectiveness of self-reflective program search for long context. *arXiv preprint arXiv:2603.15653*, 2026. URL <https://arxiv.org/abs/2603.15653>.
- [8] Amartya Roy, Rasul Tutunov, Xiaotong Ji, Matthieu Zimmer, and Haitham Bou-Ammar. The Y-combinator for LLMs: Solving long-context rot with λ -calculus. *arXiv preprint arXiv:2603.20105*, 2026. URL <https://arxiv.org/abs/2603.20105>.
- [9] Elias Lumer, Sahil Sen, Kevin Paul, and Vamse Kumar Subbiah. Recursive agent harnesses. *arXiv preprint arXiv:2606.13643*, 2026. URL <https://arxiv.org/abs/2606.13643>.
- [10] Lilian Weng. Harness engineering for self-improvement. Lil’Log, 2026. URL <https://lilianweng.github.io/posts/2026-07-04-harness/>.
- [11] Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. Recursive harness self-improvement. *arXiv preprint arXiv:2607.15524*, 2026. URL <https://arxiv.org/abs/2607.15524>.
- [12] Akshat Gupta, Jermaine Lei, Alexander Lu, Gopala Anumanchipalli, and Leshem Choshen. Automated discovery has no universally superior harness. *arXiv preprint arXiv:2607.18235*, 2026. URL <https://arxiv.org/abs/2607.18235>.
- [13] OpenAI. GraphWalks: A multi-hop long-context reasoning benchmark. Hugging Face dataset, 2025. URL <https://huggingface.co/datasets/openai/graphwalks>. Accessed July 2026.
- [14] Amanda Bertsch, Adithya Pratapa, Teruko Mitamura, Graham Neubig, and Matthew R. Gormley. Oolong: Evaluating long context reasoning and aggregation capabilities. *arXiv preprint arXiv:2511.02817*, 2025. URL <https://arxiv.org/abs/2511.02817>.
- [15] Shuai Shao, Qihan Ren, Chen Qian, Boyi Wei, Dadi Guo, Jingyi Yang, Xinhao Song, Linfeng Zhang, Weinan Zhang, Dongrui Liu, and Jing Shao. Your agent may misevolve: Emergent risks in self-evolving LLM agents. *arXiv preprint arXiv:2509.26354*, 2025. URL <https://arxiv.org/abs/2509.26354>.
- [16] Yingzhi Mao, Chunkang Zhang, Junxiang Wang, Xinyan Guan, Boxi Cao, Yaojie Lu, Hongyu Lin, Xianpei Han, and Le Sun. When models outthink their safety: Unveiling and mitigating self-jailbreak in large reasoning models. *arXiv preprint arXiv:2510.21285*, 2025. URL <https://arxiv.org/abs/2510.21285>.

310 [17] Liwei Jiang, Kavel Rao, Seungju Han, Allyson Ettinger, Faeze Brahman, Sachin Kumar, Niloo-
311 far Mireshghallah, Ximing Lu, Maarten Sap, Yejin Choi, and Nouha Dziri. WildTeaming at
312 scale: From in-the-wild jailbreaks to (adversarially) safer language models. *arXiv preprint*
313 *arXiv:2406.18510*, 2024. URL <https://arxiv.org/abs/2406.18510>.

A Fine-tuned RLM baseline (F1)

F1 reproduces the weight-training arm of Zhang and Khattab [3] so that harness optimization and weight optimization are compared on the same backbone, harness, and environment. The recipe below follows their reported setup; any deviation forced by our compute allocation will be recorded before training begins.

Backbone. Qwen3-30B-A3B-Instruct-2507, the model Zhang and Khattab [3] RL-train inside an RLM harness, and the same backbone used for B1, H1, and SH-RLM so that F1 differs from B1 only in its weights.

Algorithm. RL with `prime-rl`: decoupled PPO with GRPO-style advantages and a KL penalty against the initial policy.

Reward. The environment’s deterministic verifier on the final answer, identical to the evaluation metric and to the promotion signal used by SELF-HARNESS, so the two optimizers are driven by the same outcome measure.

Rollouts. Batch size 64 with 4 rollouts per sample.

Steps. 150 optimization steps for the length-generalization setting, with evaluation every 10 steps; Zhang and Khattab [3] run 500 steps, evaluating every 20, for their cross-domain strategy experiments.

Training inputs. Short instances only, at 8k–64k tokens, drawn from the same source-environment short split used for harness optimization. Long instances are never trained on.

Evaluation. The frozen checkpoint is evaluated on the same four untouched test sets as B1, H1, and SH-RLM, at 256k–2M tokens on the long splits.

Hardware. 8×H100 nodes.

Two conditions govern whether F1 is reported at all. If the published checkpoint is obtainable, we evaluate it directly rather than retraining, and say so. If neither the checkpoint nor the compute for training is available, F1 is dropped and its absence is reported alongside the published figures for the same environment and backbone. F1 is budgeted separately from the cost estimate of section 4, which covers the fixed-weight conditions only.

B Ablations

Sub-verification. Sub-verification enters the loop only through weakness mining, so we can withhold it without changing anything else. We repeat the full optimization run with the sub-verifier signal removed, giving the proposer root outcomes and traces alone, and compare the resulting frozen harness against SH-RLM on all four test sets. The comparison asks whether checkable child-level evidence is what makes mined failure attributions actionable, or whether the proposer recovers the same edits from traces without it. Because this doubles optimization cost, it is a preregistered secondary run on the source environment, contingent on the pilot’s cost estimate holding.

Leave-one-edit-out. Each promoted edit is removed from the final harness individually and the affected test conditions are re-evaluated. These ablations determine which edits are responsible for the final gains and whether improvements arise from the accumulated harness rather than unrelated sampling variation.

C Assessing Alignment Drift (Optional)

Given prior work on misevolution in self-optimizing agents [15], and in particular the finding that capability-driven workflow optimization can sharply reduce refusal rates on harmful requests despite passing task-level validation [15], an optional extension is to evaluate SH-RLM for alignment drift as a side effect of harness promotion. Because our promotion gate, like that of Zhang et al. [2], is defined only over verifier accuracy and cost, edits that are capability-positive but compliance-increasing (e.g., “always commit to an answer,” suppressed hedging, or reasoning-mode changes, which are known to shift safety behavior in the Qwen3 family [16]) would be promoted without

361 detection. The extension freezes a small safety probe of a few hundred harmful and borderline
362 prompts [17], runs it through the RLM under B1 and under each promoted harness in the lineage,
363 and scores refusal and harmful-answer rates with an automatic judge, yielding a safety trajectory
364 plotted alongside the accuracy trajectory. At roughly 300 prompts across ~ 8 harness states, with
365 probe runs far shorter than benchmark instances, this adds on the order of 10^7 – 10^8 tokens, well under
366 2% of the budget in section 4. It would enable us to assess whether the propose–validate–accept
367 structure incidentally preserves alignment, as its bounded edit surfaces might suggest, or whether
368 the harness pathway exhibits the same drift documented for workflow evolution; either outcome
369 is informative, and to our knowledge neither has been measured for SELF-HARNESS-style harness
370 optimization. Because no promotion decision depends on the probe, it is purely observational and
371 does not alter the preregistered optimization loop.